

Cross-Cutting Concerns Assignment

Detailed Analysis: Why and Why Not

Logging

Chosen Pattern: Decorator (injected via Dependency Injection).

Why:

- You need a specialized logger that intercepts the payload and scrubs it before it reaches storage. Because the payment provider returns full card details and your logs are stored in plain text on S3, writing raw data is a catastrophic security risk, especially since GDPR applies to your EU-based users. A Decorator wrapped around your standard logger can sanitize strings (masking CC numbers and PII) right before execution.

Why Not Others:

- Trap A (API Gateway Middleware): Logging everything automatically at the gateway means unmasked API responses (including card details) go straight to plain-text storage.
- Trap B (AOP on every method): Using AOP on every method in the Payment Service creates massive performance overhead at 50,000 requests per minute and risks logging sensitive local variables.

Error Handling

Chosen Pattern: Middleware (for boundary exception catching) and Decorator (Circuit Breaker for external calls).

Why:

- You need API-level Middleware to catch unhandled exceptions, generate a unique correlation ID, and return it so legal has a reference number for every failed transaction without showing generic messages. Additionally, you need a Circuit Breaker (often implemented as a Decorator/Proxy) around the Inventory Service; if it gets slow, the seat map must remain responsive.

Why Not Others:

- Trap D (Same middleware for all services): Using the same error handling middleware everywhere ignores service-specific failure domains. For instance, a payment provider timeout (2% of requests) leaves the transaction in an unknown state. A generic middleware would just throw a 500 error, whereas the Payment Service requires specialized logic (like marking it "pending" and polling Stripe) to handle the ambiguity.

Caching

Chosen Pattern: AOP (Aspect-Oriented Programming).

Why:

- The concert venue layout (sections, rows, seats) never changes. Caching this statically via an AOP annotation (like `@Cacheable`) on the repository layer keeps the performance high and the business logic clean.

Why Not Others:

- Middleware caching for dynamic data: You cannot broadly cache the seat map or purchase history. The seat map needs to update in near real-time to avoid the platform's #1 complaint: seeing an available seat and then being told it is gone. Furthermore, purchase history must always reflect the actual database state, meaning caching there is strictly prohibited.

Notifications

Chosen Pattern: Message Queue (Asynchronous processing).

Why:

- The notification must not block the checkout from completing. By pushing a "checkout_complete" event to a message queue, the Notification Service can process the emails and SMS independently. This also satisfies the constraint that if a notification fails, the ticket is still valid.

Why Not Others:

- AOP / Decorator / Middleware: Using any synchronous interception pattern to trigger the notification would mean the Checkout Service has to wait for the Notification Service (and subsequently the Email/SMS providers) to respond. This blocks the checkout flow and will cause cascading failures under the 50,000 req/min load.

Concern	Pattern Chosen	Why	What You Rejected	Why You Rejected It
Logging	Interceptor / Decorator (with PII Redaction)	Allows us to intercept the Stripe response before it hits the log stream to scrub PCI/GDPR data. High-volume S3 logging needs a buffer.	API Gateway Logging (Trap A)	Raw Gateway logs would capture plain-text credit card details, violating PCI-DSS and GDPR. You cannot "undo" a plain-text log on S3 easily.
Error Handling	Circuit Breaker + Idempotency Keys	Use a unique <code>transaction_id</code> (the legal reference) to query Stripe's API for the status of a timed-out request before retrying or failing.	Generic Middleware (Trap D)	Global middleware can't handle the "unknown state" of a payment timeout. It would just return an error, potentially double-charging the user on retry.
Caching	Hybrid: Distributed (Redis) & Client-side	Redis for real-time seat availability (TTL < 2s). Local/In-Memory for venue layout (static). Auth tokens in Redis to avoid DB hits.	Write-Through Cache for History	Purchase history must be Strongly Consistent . Caching it risks showing a "failed" status when it actually succeeded, or vice versa.
Notifications	Message Queue (Asynchronous)	Checkout service pushes a "Success" event to a queue (RabbitMQ/SQS). Notification service consumes it. Non-blocking and resilient.	Synchronous API Calls	If the SMS provider is slow, it would hang the Checkout Service, causing the 50k RPM load to cascade and crash the whole system.